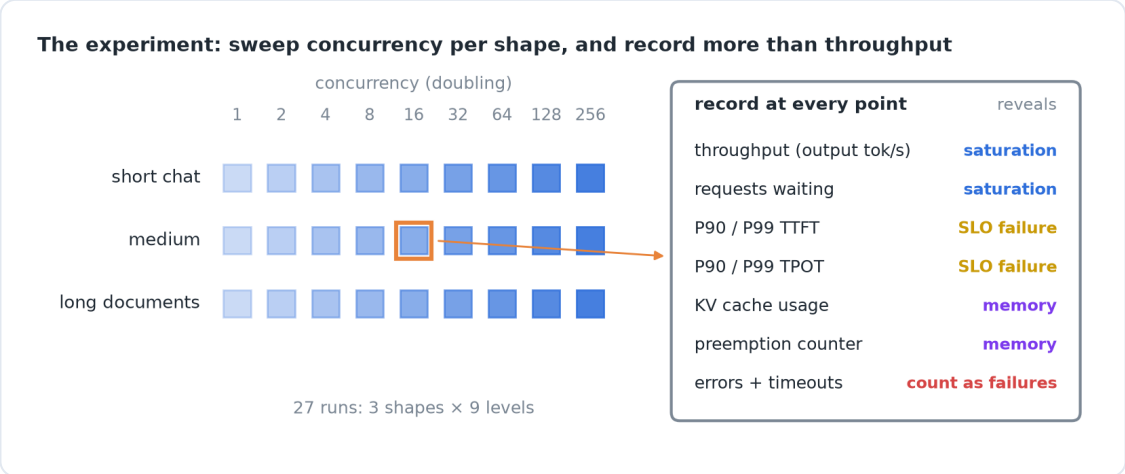
27 · Find Saturation, SLO Failure & OOM Boundaries

Source: fanout.sh / inference-eng / 05-05 · ~6 min

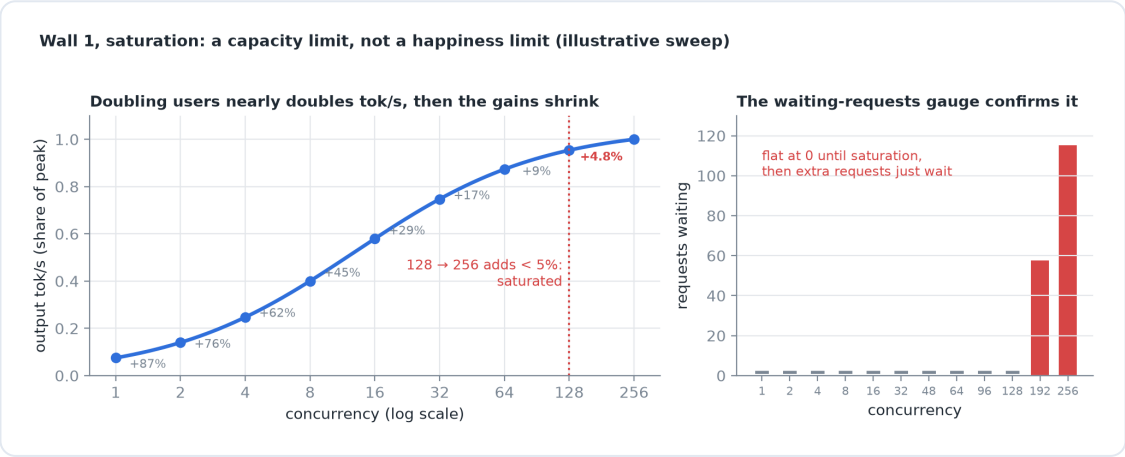
> **IN SHORT**
>
> Every GPU serving a model has an **operating envelope**: the region where it's fast, correct and stable. Outside it, it breaks against three different walls: **saturation** (throughput stops growing), **SLO failure** (the tail crosses the target) and **memory** (startup refusal, preemption, CUDA OOM). The SLO wall usually comes first, so **real capacity is the first SLO crossing**, not the throughput plateau. The memory wall shows up as **preemption before it shows up as a crash**; cap `max_num_seqs` so extra requests wait in a visible queue.

1. The experiment



Pick a few shapes that match your traffic (short chat, medium, long documents) and sweep concurrency 1, 2, 4, … up to a few hundred for each. At every point record throughput, P90 and P99 of TTFT and TPOT, three engine signals (KV cache usage, the preemption counter, requests waiting), and errors and timeouts counted as failures. Those columns are enough to find every wall.

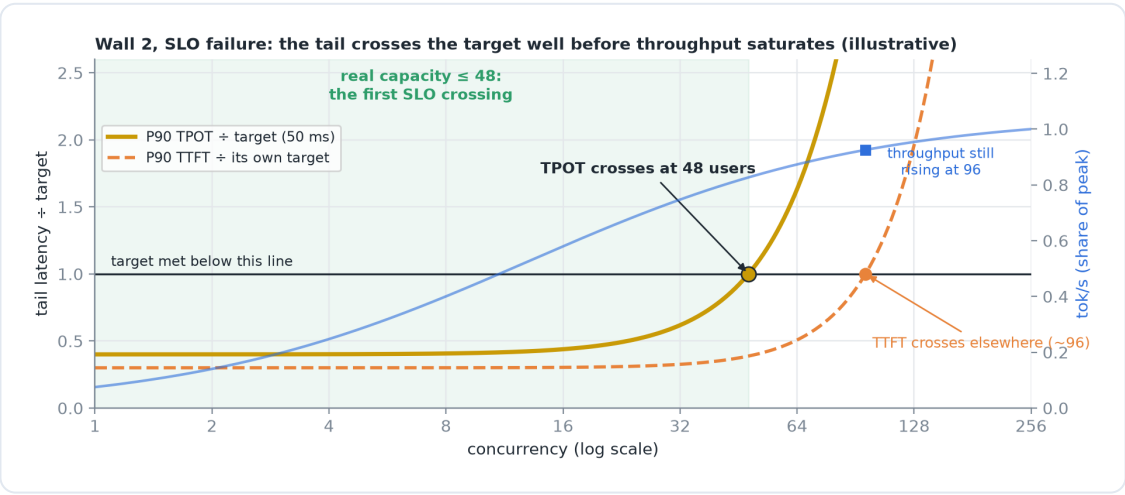
2. Wall 1: saturation



At first, doubling users nearly doubles tokens/s; then the gains shrink and the curve flattens.

- Rule: once **doubling concurrency adds less than ~5%**, you've saturated.
- Past that, extra requests aren't served faster, they **wait**: the **waiting-requests gauge**, flat at 0, starts to climb.
- Saturation is a **capacity limit**: the most this GPU can do, not whether users are happy.

3. Wall 2: SLO failure

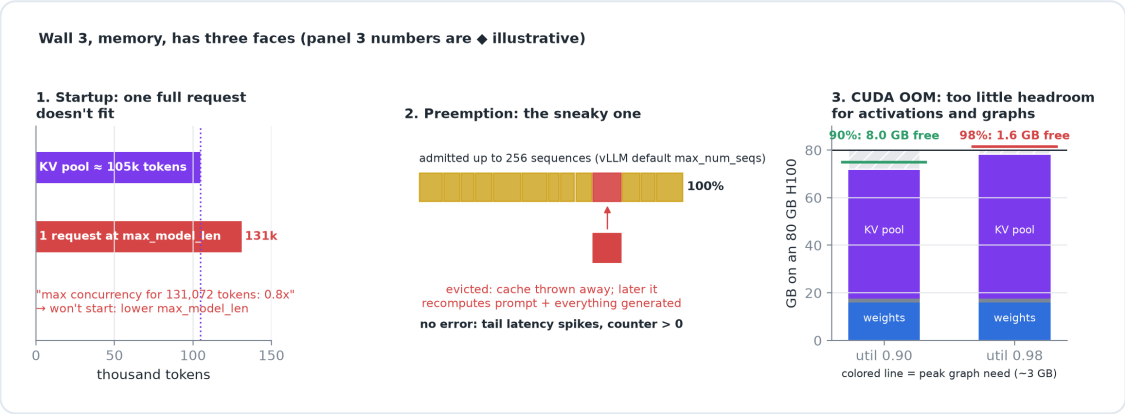


P90 TPOT rises slowly, bends up and crosses its 50 ms target at about 48 users. TTFT may cross somewhere else. Throughput is still climbing at 96.

KEY IDEA

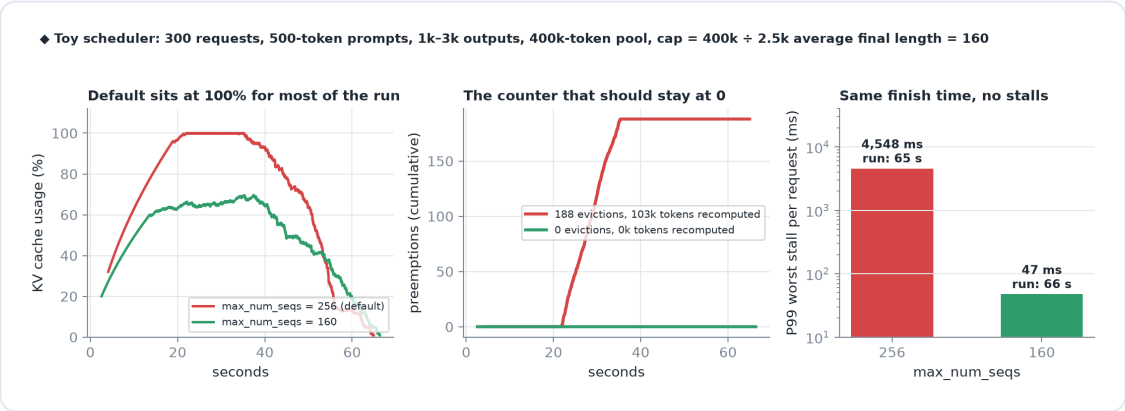
The SLO usually breaks **well before** throughput saturates. Your real capacity is the smaller of the two: the **first SLO crossing**, not the plateau.

4. Wall 3: memory, three faces



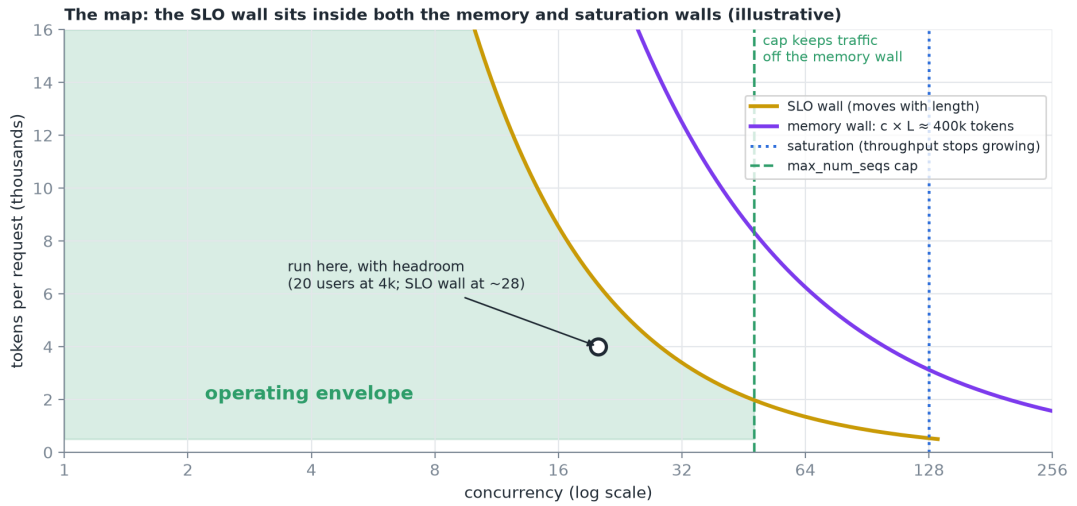
Startup: if weights plus one full-length request don't fit, the engine refuses to start or reports that less than one request fits. Preemption: the engine admits many requests because their caches start small; when the cache hits 100% it evicts one. CUDA OOM: a real crash, usually from leaving too little headroom for activations and graphs.

- **Preemption is the sneaky one.** vLLM admits up to **256 sequences** by default (`max_num_seqs`). As replies grow the cache fills; at **100%** it evicts a running request, by default **throwing its cache away** and later **recomputing prompt plus everything generated so far**.
- That request stalls, and the recompute steals time from everyone. **No error**: just tail-latency spikes and a **preemption counter** that should have stayed at 0.
- **Fix:** cap `max_num_seqs` at what the cache really sustains, so extra requests wait in a **visible queue** instead of thrashing.



◆ *A toy scheduler: with the default cap the cache sits at 100%, 188 evictions recompute 103k tokens and the P99 worst stall is 4.5 s. Capping at 160 finishes in the same time with no evictions and a 47 ms worst stall.*

5. The operating envelope



Concurrency across, tokens per request up. The memory wall is a curve: concurrency × length can't exceed the cache, ~400k tokens here. Saturation is where throughput stops growing. The SLO wall moves with length and sits inside both. Run inside the green region with headroom for bursts, and set the concurrency cap so traffic can't wander past the memory wall.

Wall	Signal that gives it away	What it means
Saturation	Doubling adds < 5%; waiting gauge leaves 0	The most this GPU can do
SLO failure	P90/P99 TTFT or TPOT crosses the target	Real capacity (usually first)
Memory	Startup refusal; preemption counter > 0; CUDA OOM	Cache or headroom too small

6. Recap

Lesson 27 on one card

THE SWEEP

every shape × concurrency 1 … 256
record tok/s, P90/P99 TTFT + TPOT,
KV usage, preemptions, queue, errors

SATURATION

doubling adds < 5% → flat
waiting gauge leaves 0
the most this GPU can do

SLO FAILURE

P90 tail crosses the target
usually first (48 vs still rising at 96)
real capacity = first crossing

MEMORY

startup refusal · preemption · CUDA OOM
preemption shows before a crash
fix: cap max_num_seqs → visible queue